Clase 245 — Gestión de vulnerabilidades a escala

> Parte: **11 — DevSecOps y seguridad del SDLC** · Fuente: *Securing DevOps* (Julien Vehent) y NIST SP 800-40r4 (Guide to Enterprise Patch Management) ⏱️ Duración estimada: **110 min** · Nivel: **Avanzado**

🎯 Objetivo

Pasar de "escanear y encontrar vulnerabilidades" a **operar un programa de gestión de vulnerabilidades**: consolidar hallazgos de muchas herramientas, deduplicarlos, priorizarlos por riesgo real, asignarlos con SLA, seguir su remediación y medir el programa con métricas. Cuando tienes cientos de repos y miles de hallazgos, sin un proceso te ahogas; con proceso, reduces el riesgo de forma medible.

📚 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Diseñar** el ciclo de vida de una vulnerabilidad (descubrir → triage → priorizar → remediar → verificar).
2. **Priorizar** con un modelo de riesgo (CVSS + EPSS + KEV + exposición + criticidad del activo).
3. **Definir** SLAs de remediación por severidad y medir su cumplimiento.
4. **Consolidar** hallazgos de múltiples fuentes evitando duplicados y fatiga.
5. **Reportar** el estado con métricas útiles para dirección y equipos.

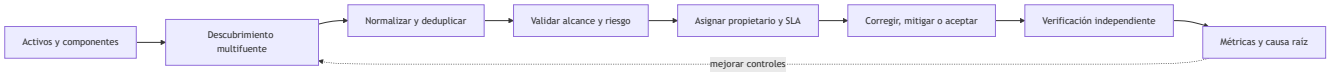
🗺️ Temas

#	Tema	Por qué importa
1	Ciclo de vida de la vulnerabilidad	El proceso, no solo el escaneo
2	Consolidación y deduplicación	Muchas herramientas, un backlog
3	Priorización basada en riesgo	No todo lo crítico es urgente
4	SLAs por severidad	Compromisos medibles de remediación
5	VEX y excepciones	Documentar lo no explotable
6	Métricas del programa	MTTR, backlog, tasa de recurrencia
7	Herramientas de agregación	DefectDojo y similares

🧠 Explicación en profundidad

Gestionar vulnerabilidades es operar decisiones a lo largo del tiempo

Un escaneo produce observaciones; un programa produce resultados. Para ello necesita inventario, propietario, deduplicación, validación, priorización, tratamiento, verificación y aprendizaje. Sin inventario no se conoce la exposición; sin propietario el ticket envejece; sin verificación, «cerrado» puede significar que alguien cambió un estado.



La deduplicación no debe borrar contexto: un mismo CVE en una estación aislada y en una API pública comparte identificador, pero no exposición ni propietario. Se conserva la instancia por activo y se agrupa para coordinar una solución. La fuente, fecha, evidencia y versión de herramienta permiten explicar por qué un hallazgo apareció o desapareció.

Priorizar con varias dimensiones

CVSS describe severidad técnica bajo supuestos; KEV aporta evidencia de explotación conocida; EPSS estima probabilidad de explotación; el negocio aporta criticidad, exposición, privilegio y controles. Una fórmula puede ordenar, pero debe ser transparente y revisable. «Crítico = 24 horas» sin capacidad de emergencia ni pruebas puede fomentar cierres falsos. El SLA empieza cuando el hallazgo es accionable y contempla rutas de excepción, escalamiento y verificación.

El tratamiento no siempre es «parchear». Puede actualizar, retirar, reconfigurar, aislar, deshabilitar una función o aceptar temporalmente con aprobación. La aceptación registra el riesgo residual, controles compensatorios y vencimiento. VEX sirve para comunicar estados de afectación de un producto, no para declarar «no afectado» sin análisis.

Métricas que no castigan la honestidad

Contar vulnerabilidades abiertas puede subir cuando mejora la cobertura. Son más útiles la edad por riesgo, tiempo hasta triage y remediación, porcentaje de activos con propietario, reaperturas, excepciones vencidas y recurrencia de causas. Una mediana sola oculta la cola; se observan percentiles y casos fuera del SLA. Las métricas deben provocar decisiones, no rankings de equipos que incentiven ocultar resultados.

Caso razonado: mil críticos y ninguna prioridad

Tras integrar escáneres, DefectDojo recibe mil hallazgos críticos duplicados. El equipo normaliza identificadores, conserva instancias por activo y separa imágenes no desplegadas. Cruza exposición y KEV, valida una ruta pública explotable y asigna un dueño. Los hallazgos heredados entran en campañas por componente; los nuevos rompen la línea base según política. El valor no provino de cerrar mil tickets, sino de convertir datos en una cola defendible.

📓 Glosario operativo

Término	Definición útil
Instancia	Aparición concreta de una vulnerabilidad en un activo o artefacto.
SLA	Compromiso temporal condicionado por riesgo y proceso definido.
Riesgo residual	Riesgo que permanece después del tratamiento.
Control compensatorio	Medida alternativa que reduce riesgo sin eliminar la causa.
Reapertura	Evidencia de que la corrección no eliminó o reintrodujo el problema.

✅ Criterio de dominio

El alumno domina el programa cuando puede convertir hallazgos heterogéneos en instancias trazables, ordenar con contexto, asignar tratamiento y vencimiento, verificar el cierre y elegir métricas que distingan cobertura, velocidad y riesgo residual.

Cómo lo nombra el catálogo CWE y qué cambia al priorizar

La gestión de vulnerabilidades a escala necesita **unidades de análisis** distintas, y el trío CWE–CVE–instancia las ordena. Un **CVE** identifica la vulnerabilidad concreta en un producto; la **instancia** es ese CVE en un activo concreto (la que tiene dueño, exposición y plazo); y el **CWE** es la causa común que agrupa muchos CVE. Esa última capa es la que permite la métrica de la clase de «recurrencia de causas»: si el 30 % de tus hallazgos son `CWE-89` o `CWE-798`, no tienes 400 tickets sino un problema de proceso (una capa de acceso a datos que no parametriza, una práctica de secretos que falla) y la intervención es una sola.

Para el plazo, ten presente el modelo de **CISA BOD 26-04** (10 de junio de 2026), que sustituye a BOD 22-01 y 19-02: obligatorio solo para agencias civiles federales de EE. UU., pero una referencia útil porque combina cuatro variables (exposición del activo, presencia en KEV, si la explotación es automatizable y si el impacto técnico es total o parcial) en una tabla de 16 filas que da plazos de 3, 14 o 60 días o «próxima actualización», y que añade un triaje forense cuando hay explotación confirmada con control total. Sobre los datos reales del programa, el mismo CVE pasa de «3 días y triaje forense» a «14 días» solo con cambiar el activo de expuesto a interno. Puedes reproducirlo con `python labs/devsecops-pipeline/priorizar.py --instantanea ... --bod-26-04`; la explicación y las limitaciones están en la clase 369.

📖 Definiciones y características

- **Gestión de vulnerabilidades**: proceso continuo de identificar, evaluar, tratar y reportar vulnerabilidades. *Característica*: es un ciclo, no un evento puntual.
- **Triage**: clasificar un hallazgo (real/falso, severidad, dueño). *Característica*: filtra ruido antes de asignar trabajo.
- **Priorización basada en riesgo (RBVM)**: ordenar por riesgo real, no solo por CVSS. *Característica*: combina severidad, explotabilidad, exposición y valor del activo.
- **SLA de remediación**: plazo máximo para corregir según severidad. *Característica*: hace el compromiso medible y auditable.
- **VEX**: Vulnerability Exploitability eXchange, documento que declara si un CVE es explotable en tu producto. *Característica*: reduce ruido justificando "not affected".
- **MTTR**: tiempo medio de remediación. *Característica*: métrica clave de la salud del programa.

🧰 Herramientas y preparación

- **DefectDojo** (open source) — plataforma de gestión y consolidación de hallazgos.
- **Trivy / Semgrep / ZAP** — fuentes de hallazgos (clases anteriores).
- **CISA KEV** y **EPSS** — señales de priorización.
- Hoja de cálculo o issue tracker (Jira/GitHub Issues) para el flujo si no usas DefectDojo.

```
# DefectDojo de práctica con Docker Compose:
git clone https://github.com/DefectDojo/django-DefectDojo
cd django-DefectDojo && docker compose up -d
```

🧪 Laboratorio guiado

> 🧪 **Laboratorio ejecutable del programa:** `devsecops-pipeline` — incluye el bloque de priorización **KEV** → **EPSS** → **CVSS** y el procedimiento de remediación con reversión automática.

Ejercicio de proceso (defensivo, sobre hallazgos propios):

1. **Recolecta hallazgos**. Exporta resultados de Trivy (SCA/imagen), Semgrep (SAST) y ZAP (DAST) de tus prácticas en formato JSON.
2. **Consolida en DefectDojo**. Crea un producto y engagement, importa los tres informes. Observa cómo deduplica hallazgos repetidos entre escaneos.
3. **Enriquece con señales de riesgo**. Para los top 20 hallazgos, añade EPSS y marca los que están en CISA KEV.
4. **Prioriza**. Ordena por un score compuesto: severidad × exposición (¿internet-facing?) × criticidad del activo, elevando los KEV/EPSS alto. Documenta el criterio.
5. **Asigna SLAs**. Define plazos, p. ej.: Crítica explotable 7 días, Alta 30, Media 90, Baja 180. Asigna dueño a cada hallazgo top.
6. **Documenta excepciones con VEX**. Para un CVE en una función que no usas, redacta un statement "not affected" con justificación.
7. **Genera métricas**. Calcula backlog por severidad, MTTR del último periodo y tasa de recurrencia (hallazgos que reaparecen). Prepara un mini-dashboard.

> Nota ética: la gestión de vulnerabilidades es puramente defensiva. Los datos de hallazgos son sensibles; trátalos con control de acceso y no los publiques.

✍️ Ejercicios

1. Dibuja el ciclo de vida de una vulnerabilidad en tu organización.
2. Consolida hallazgos de dos herramientas y cuenta los duplicados eliminados.
3. Prioriza 10 hallazgos con un modelo que combine CVSS, EPSS y KEV.
4. Define una tabla de SLAs por severidad y justifica los plazos.
5. Redacta un statement VEX "not affected" para un CVE concreto.
6. Calcula el MTTR de un conjunto de hallazgos con fechas de apertura/cierre.

📝 Reto verificable

Opera un mini-programa de gestión de vulnerabilidades sobre hallazgos reales de tus prácticas.

Criterio de aceptación: (a) los hallazgos de al menos tres herramientas están consolidados y deduplicados; (b) existe un modelo de priorización documentado que va más allá de CVSS (incluye EPSS/KEV/exposición); (c) hay SLAs por severidad y cada hallazgo top tiene dueño y plazo; (d) al menos una excepción está documentada como VEX; y (e) se reporta MTTR y backlog por severidad.

⚠️ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Backlog de 10.000 hallazgos sin tocar	Priorizas por CVSS crudo. Filtra por explotabilidad y exposición; ataca lo alcanzable primero.
El mismo bug aparece 30 veces	No hay deduplicación. Usa una plataforma (DefectDojo) que fusione hallazgos equivalentes.
Los equipos ignoran los tickets	Sin SLA ni dueño claro. Asigna responsable y plazo, y escala incumplimientos.
Se remedia pero reaparece	No se corrigió la causa raíz (dependencia base). Mide tasa de recurrencia y ataca el origen.
Métricas que nadie usa	Reportas número de CVE sin contexto. Mide MTTR, backlog por riesgo y tendencia.

❓ Preguntas frecuentes

❓ **¿Por qué no arreglar simplemente todo lo "crítico"?** Porque "crítico" en CVSS no significa "explotable en tu contexto". Un crítico sin exploit y sin exposición puede esperar frente a un alto que está en KEV y es internet-facing.

❓ **¿Qué SLAs son razonables?** Depende del riesgo y del sector, pero un patrón común: crítico explotable en días, alto en semanas, medio en meses. Lo importante es que sean explícitos y medibles.

❓ **¿Necesito DefectDojo o basta con Jira?** Para pocos repos, un tracker con buena disciplina sirve. A escala, una plataforma que importe múltiples formatos, deduplique y calcule métricas ahorra muchísimo esfuerzo.

❓ **¿Qué es y para qué sirve VEX?** Es una forma estandarizada de decir "este CVE está presente pero no somos explotables porque...". Reduce el ruido y evita remediar lo que no aplica, con trazabilidad.

🔗 Referencias

- 🏢 **En la empresa: Qualys / Tenable / Rapid7** para el escaneo a escala y **DefectDojo** para orquestar, deduplicar y dar seguimiento a los hallazgos.
- NIST SP 800-40r4 Patch Management — https://csrc.nist.gov/pubs/sp/800/40/r4/final
- DefectDojo — https://www.defectdojo.org/
- CISA KEV — https://www.cisa.gov/known-exploited-vulnerabilities-catalog
- EPSS — https://www.first.org/epss/
- CycloneDX VEX — https://cyclonedx.org/capabilities/vex/

📥 Material descargable

- 📄 Guía en PDF — versión imprimible de esta clase.
- 🎞️ Presentación (PPTX) — deck para proyectar en clase.

⬅️ Clase anterior

Clase 244 — Políticas como código con OPA

➡️ Siguiente clase

Clase 246 — Supply chain security: SBOM y SLSA